



Testing Perspectives

د. سعيد أبو طراب

محتوى مجاني غير مخصص للبيع التجاري

هندسة البرمجيات (2)

RB Informatics;

تحدثنا أنه يوجد لدينا أنواع مختلفة لعملية الـ Testing خلال مرحلتي الـ Construction و Deployment وهي: User, Release, Development

1. Development Testing: هي الاختبارات التي تتم خلال عملية التطوير Construction أو بشكل عام هي الاختبارات التي تتولد مع بداية المشروع وخلال هذه المرحلة يتم العمل بطريقة تزايدية Incremental وبالتالي سيتم الاختبار على مستوى الـ Unit ثم Integration ثم System.
2. Release Testing: هو فحص المنتج البرمجي قبل الإطلاق لضمان أنه يلبي المتطلبات وتعتبر صلة الوصل بين مرحلتي Construction و Deployment.
3. User Testing: وهي اختبار التطبيق من قبل المستخدم وبالتالي يكون الاختبار بناءً على الطريقة التقليدية للاستخدام ورغبة المستخدم.

Testing Perspectives

والمقصود بها عندما نقوم بفحص أي شيء إما نقوم بفحصه مع معاينة بنيته الداخلية أو نقوم بفحصه بناءً على وظائفه فقط وبالتالي هناك منظورين للاختبار البرمجي وهما White Box و Black Box.

Black Box Testing

فحص البرمجيات وظيفياً من وجهة نظر المستخدم، أي بدون رؤية الـ Structure أو المكونات الداخلية أي الـ Code حيث يعتمد تقييم الأداء على مدخلات البرنامج ونتائج الإخراج.

White Box Testing

يكون عكس اختبار Black Box حيث يتم اختبار البرمجيات بشكل مفصل من خلال دراسة الـ Code الخاص بالبرنامج.

- يمكن أن نفترض أن الـ White Box Testing يحصل في مرحلة الـ Development Testing بينما الـ Black Box Testing يحصل على مستوى User Testing أو Release Testing.
- أصغر وحدة برمجية يمكن اختبارها هي الـ Unit حيث في البرمجة الإجرائية تعد الـ Unit هي الـ Function وفي البرمجة غرضية التوجه OOP تعد الـ Unit هي الـ Object.
- يوجد لدينا Tester يهتم باختبار الوظائف التي علينا تحقيقها من وجهة نظر المستخدم ويوجد آخر يهتم باختبار المكونات الداخلية أي الـ Code.

يمكن اختبار البرنامج من وجهة نظر ال Tester من خلال:

- 1. تحديد Test Cases مناسبة تؤدي إلى اكتشاف الأخطاء.
- 2. بناء Testing Strategy/Policy: أي يجب أن يكون لدينا معيار يحدد لماذا قمنا باختيار هذا ال Test و متى علينا التوقف عن عملية الاختبار.

مثال: كود لحل معادلة من الدرجة الثانية:

```
1. double a, b, c, d, x1, x2;
2. d = b*b - 4(a*c);
3. if (d >= 0) {
4.     x1 = (-b + sqrt(d)) / 2*a;
5.     x2 = (-b - sqrt(d)) / 2*a;
6.     cout<<x1<<x2};
7. else
8.     cout<<"no solution";
9. return 0;
```

المدخلات الخاصة بهذا التابع هي a, b, c والمخرجات هي x1, x2.

نولد حالات اختبار مختلفة من الشكل: (1, 1, 1) (مستحيلة الحل) أي a=1, b=1, c=1

(2, 2, 2) (مستحيلة الحل)، (3, 3, 3) (مستحيلة الحل)، (1000, 1000, 1000) (مستحيلة الحل)...

بناءً على هذه الاختبارات العشوائية هل نكون قد قمنا بفحص ال Code بشكل صحيح؟!

فعليا لا، لأننا بجميع الحالات قمنا بالفحص بدون معرفة الحالة البرمجية التي تؤدي إلى هذه النتيجة. إذا نظرنا إلى ال Code و تتبعنا مسار حالات الاختبار سنجد أننا قمنا بفحص نفس المسار بجميعها و التي تؤدي إلى قيمة دلّتا سالبة، وباقي المسارات التي تؤدي إلى دلّتا أكبر أو تساوي الصفر لم نقم بفحصها. لذا تبين لدينا أنه رغم تطبيقنا لمجموعة من ال Test Cases لم نقم بفحص كامل ال Code، بل قمنا بفحص Segment معين منه فقط.

نتيجة: لمعرفة اختيار ال Test Cases المناسبة والتي تعطي معامل تغطية محدد يجب علينا أن نكون قادرين على تخيل ال Code وطريقة سيره.

بملاحظة ال Code يجب علينا تتبع مسارات كل دخل و لكن هذا سيتطلب الكثير من الوقت و الجهد خاصة في حالات ال Codes الكبيرة لذلك الأسهل أن نمثل ال Code ك Flow و ذلك باستخدام CFG.



وعلى امتداد الدرب للهدف الذي تسعى له لا تفعل

التشويقا

أحلى من الأحلام قبل وقوعها شغف بها يدني لها

التحقيقا

CFG: Control Flow Graph

هنا سندمج الفكرة التصميمية والتنفيذية، حيث كل سطر ضمن code سنقوم بتمثيله ب node أيًا كان ما يحويه حيث ما يهمنا هو ال flow الخاص بالcode وليس كل statement وما تمثله، وهذا عكس مخطط flow chart الذي يهتم بالأشكال بناء على محتوى كل statement.

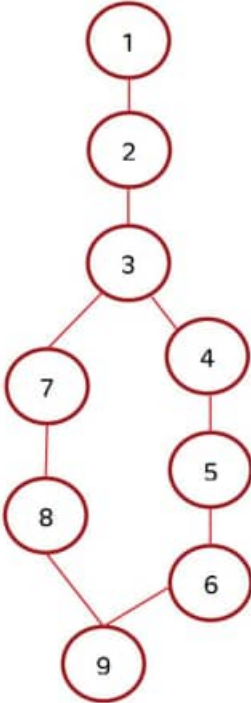
عندما تكون ال node خارج منها فرعين "2 branches" فهي predicate:

- أول نقطة تسمى input case
- آخر نقطة تسمى output case

نمثل الcode السابق كـ CFG بحيث نعطي لكل node رقم السطر

فيكون الشكل:

نلاحظ أن جميع ال Test Cases اتبعت المسار (123789).
ومنه أصبح لدينا رؤية أفضل لما قمنا باختباره و ما لم نختبره.



Coverage Criteria

في معاملات التغطية في ال Testing نحن نقوم باختيارها ضمن ال Policy حسب ال Application فهي تعتبر مقياس للتقدم وبالتالي يجب تحديد معاملات التغطية وفقا للمشروع حيث كلما كان احتمال فشل التطبيق أكبر كلما اخترنا التغطية أكبر وبالتالي حسب نوع التطبيق نختار ال Coverage ومما سبق نستنتج إذا أردنا إجراء اختبارات أكثر علينا اختيار معامل تغطية أكثر دقة.

- **أنواع معاملات التغطية:**

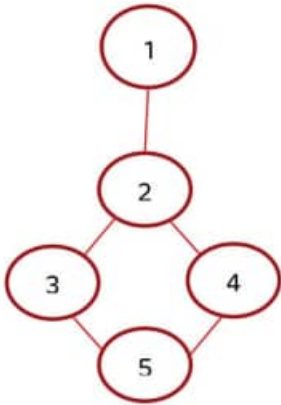
1. Statement Coverage: كل سطر برمجي ضمن الكود هو عبارة عن Statement وبناء على نسبة التغطية المطلوبة من Statement Coverage يتم زيادة عدد ال Test Cases وبالتالي تحقيق زيادة عدد أكبر من الأسطر البرمجية

أي في حال كانت النسبة المطلوبة هي 100% عندئذ يجب أن يتم زيارة جميع الأسطر البرمجية التي تحقق البرنامج (زيارة جميع العقد في ال CFG).

2. Branch Coverage: كل شرط عبارة عن Branch وبالتالي المطلوب توليد أقل عدد من ال Test Cases بحيث تحقق زيارة أكبر عدد ممكن من ال Branches.

3. Path Coverage: وهو عبارة عن تسلسل الانتقال من نقطة الدخول إلى نقطة الخرج.

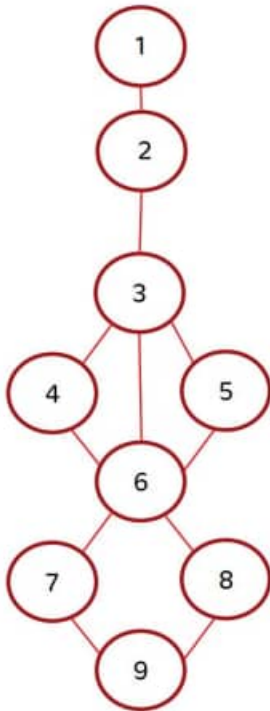
Test Case عبارة عن دخل input و الخرج المتوقع expected output و التي تقابل على ال (CFG) Graph مسار نسميه Testing Path.



Test path_1: 1 2 3 5

Test path_2: 1 2 4 5

أي أن ال Test Path يبدأ من أول نقطة وينتهي بأخر نقطة.

من هو معامل التغطية الأدنى أو الذي يولد اختبارات أكثر؟

للإجابة على السؤال سنحيط عن الأسئلة التالية بناء على المثال التالي:

كم testing path نحتاج لنحصل على Statement Coverage؟

نحتاج إلى مسارين لتغطي العقد التسعة:

path_1: 1 2 3 5 6 8 9

path_2: 1 2 3 4 6 7 9

كم testing path نحتاج لنحصل على Branch Coverage؟

نحتاج إلى 3 مسارات لتغطي الأفرع الثلاثة:

path_1: 1 2 3 5 6 8 9

path_2: 1 2 3 4 6 7 9

path_3: 1 2 3 6 7 9

ومما سبق نستنتج أن:

معامل تغطية ال Branch يعطي اختبارات أكثر أي هو أدق من ال Statement وبالتالي عندما نحقق ال Branch Coverage نكون قد حققنا ضمناً ال Statement Coverage.

كم testing path نحتاج لنحصل على Path Coverage؟

نحتاج إلى 6 مسارات لتغطيتها جميعها وهي:

path_1: 1 2 3 4 6 7 9

path_2: 1 2 3 4 6 8 9

path_3: 1 2 3 6 7 9

path_4: 1 2 3 6 8 9

path_5: 1 2 3 5 6 7 9

path_6: 1 2 3 5 6 8 9

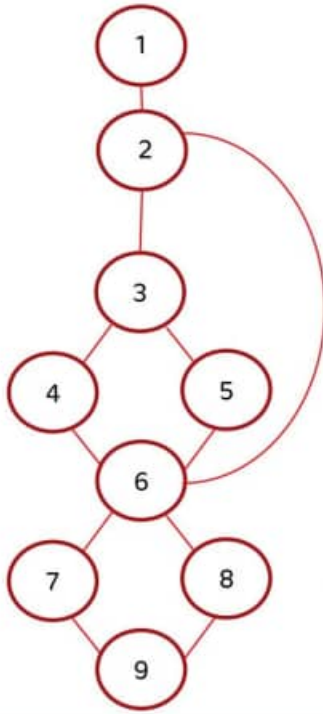
ومما سبق نستنتج أن:

معامل تغطية ال Path هو الأدق وحقق ضمناً ال Branch + Statement.

من هو معامل التغطية الأدق أو الذي يولد اختبارات أكثر؟

نلاحظ أنه ليس كل Statement لفحصها نحتاج إلى test أو استطيع خلق test case لها لأن ال Statement قد لا يكون لها دخل وخرج محددين.

للإجابة على السؤال سنجيب عن الأسئلة التالية بناء على المثال التالي:

**كم testing path نحتاج لنحصل على Statement Coverage؟**

نحتاج إلى مسارين لتغطي العقد التسعة: (1234689) و (1235679)

كم testing path نحتاج لنحصل على Branch Coverage؟

نحتاج إلى 3 مسارات لتغطي الأفرع الثلاثة: 12689 بالإضافة إلى مسارات ال Statement Coverage.

كم testing path نحتاج لنحصل على Path Coverage؟

نحتاج إلى 6 مسارات لتغطيتها جميعها وهي:
(1234679) و (1234689) و (1235679) و (1235689) و (12679) و (12689).

- قد نصل إلى Infeasible Path أي مسار غير قابل للتحقيق ونقوم دائماً بالتحقق من وجوده لاستثنائه.
- في حال ظهر لدينا Infeasible Path فإن نسبة معامل التغطية لا تتغير (أي النسبة المئوية تعتمد على Feasible Path).
- في بعض الحالات قد يؤدي وجود loop إلى عدم القدرة على تحديد ال Testing Path أي يصبح عدد ال paths لا نهائي وبالتالي من المستحيل القول أنه تم تحقيق 100% من ال Path Coverage لذلك في حال وجود حلقة سنواجه مشكلة الانفجار بعدد المسارات.

لحل مشكلة الانفجار بعدد المسارات هناك عدة طرق:**1. تحديد نسبة التغطية المقبول الوصول إليها:**

يقوم ال tester بتحديد ال coverage criteria ضمن ال test policy، وتزيد صعوبة هذه العملية مع زيادة تعقيد الكود.

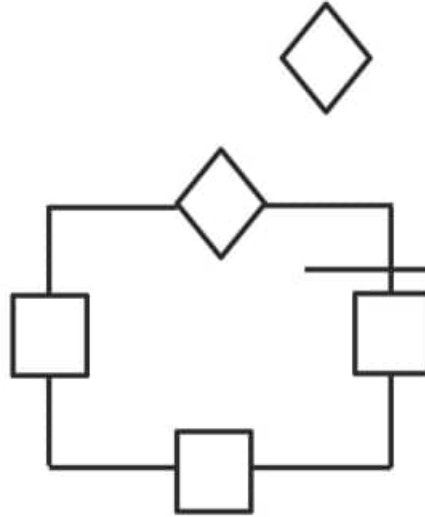
نقول أننا نحقق نسبة تغطية 100% عندما نستطيع إيجاد حالة اختبار واحدة على الأقل لكل مسار.
مثال: عندما يكون لدينا 4 مسارات وأوجدنا حالة اختبار واحدة لكل مسار عندها نكون حققنا تغطية بنسبة 100% وفي حال أوجدنا 3 حالات اختبار للأربع مسارات نكون قد حققنا نسبة تغطية 75% وهكذا.

2. التركيز على ال Basic Path:

في حال وجود loops ضمن الاختبار سيكون لدينا عدد لا نهائي من ال testing paths، وفي هذه الحالة بدلاً من إيجاد ال path coverage يمكن إيجاد أهم ال Paths التي يمكن من خلالها إيجاد ال test case.

Basic path: هو المسار الذي يؤثر على الـ complexity الخاص بالنظام.

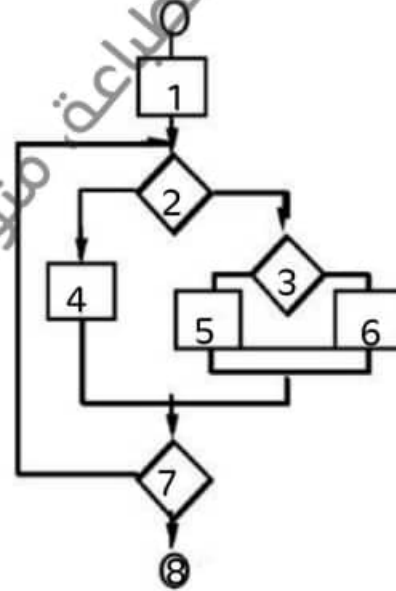
ولكي نوجد المسارات الأساسية لدينا قاعدة الـ cyclomatic complexity، وهي تساعدنا بعد إيجاد الـ CFG على إيجاد عدد المسارات الأساسية و ذلك بأحد الطرق:



1. عدد الشروط + 1 (عدد المعينات في الرسم).
2. عدد المناطق المغلقة + 1
3. $Edges - Nodes + 2 * p$

Path_1 = 1, 2, 3, 6, 7, 8
 Path_2 = 1, 2, 3, 5, 7, 8
 Path_3 = 1, 2, 4, 7, 8
 Path_4 = 1, 2, 4, 7, 2, ..., 7, 8

تذكير: الـ Testing Path هو مجموعة وصلات بدءاً من نقطة الدخول إلى نقطة الخرج.



Cyclomatic Complexity = 4

نتيجة: يمكن إيجاد الـ Testing Path في حالات الـ loops عن طريق إيجاد الـ Basic Paths و التي يمكن أن نوجد عددها عن طريق القاعدة السابقة.

الحلقة في المثال السابق ستسبب مشكلة انفجار المسارات. تمثل الحلقة فعلياً بمسار واحد في قاعدة الـ basic paths.

في حال وجود وقت كافٍ للاختبار مسارات الحلقة نقوم بـ:

- اختبار نفس المسار بداتاً مختلفة (حيث يحكم الدخول إلى الحلقة قيمة المتغير في شرط الحلقة).
- اختبار أكثر من مسار بداتاً مختلفة.


```

1. Input (x, y);
2. z = 0;
3. while (x + y > 0) {
4.     if (x > 0)
5.         x = x - 1;
6.     If (y > 0)
7.         Y = y - 1;
8.     z = z + x + y;
9. }
9. output (z);

```

مثال آخر: ليكن لدينا code التالي، والمطلوب:

1. رسم مخطط CFG لل code.
2. إيجاد testing paths for statement coverage.
3. إيجاد testing paths for branch coverage.
4. إيجاد testing paths for path coverage.
5. إيجاد test case لكل path.

الحل:

1. لرسم ال Graph نلاحظ أن العقدة 1 تمر على التسلسل مع 2

و 2 تمر على التسلسل مع 3 وتبدأ عملية التفرع عند العقدة 3.

نعلم أن الحلقة عبارة عن repeated condition وبالتالي

لدينا فرع في حال لم يتحقق الشرط يذهب إلى 9 و يطبع الخرج،

أما في حال التحقق سيدخل إلى الحلقة.

لدينا ضمن الحلقة شرطين الأول عند العقدة 4 و الثاني عند العقدة 6.

في حال تحقق الشرط عند العقدة 4 سيذهب إلى 5

وإذا لم يتحقق سيذهب إلى 6، وفي حال تحقق الشرط عند العقدة 6

سيذهب إلى 7 و إذا لم يتحقق سيذهب إلى 8.

في العقدة 8 سنعود إلى العقدة 3 للتحقق من شرط الحلقة مرة أخرى.

2. نجد أن مساراً واحداً غطى كل العقد:

Statements = 9

Testing Path (1): 1234567839 => test case (1,1,0)

أي أننا سنمر في هذا ال Path إذا كان الدخل $x = 1, y = 1$ وسيكون الخرج

(وهو في مثالنا المتحول z) قيمته 0

3. نحتاج إلى ثلاثة مسارات لتغطية كل الفروع:

Branches = 3

(عند العقدة 3 و 4 و 6)

Testing Path (3): 1239 => test case (1, -1,0)

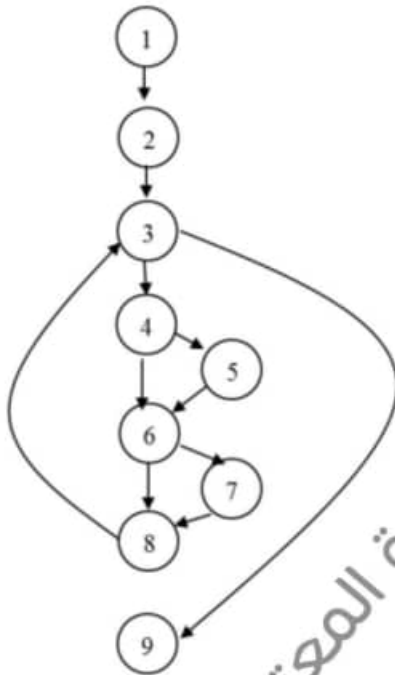
1234567839 => test case (1,1,0)

12346839

يمكننا نظرياً بمسارين أن نحصل على branch coverage

تذكرة:

Test Case(input, expected output)



4. بما أنه يوجد لدينا حلقة نستطيع تجريب عدد لا نهائي من الحالات وبالتالي سنواجه مشكلة path explosion أي انفجار بعدد المسارات أي لا نستطيع تحقيق path coverage.

- Path = لا نهائي ,
- (عدد المسارات التي يمكن اختبارها بغض النظر عن الحلقة) testing path = 5
- 1239 => test case (0, 0, 0)
- 1234567839 => test case (1, 1, 0)
- 123467839 => test case (0, 1, 0)
- 123456839 => test case (1, 0, 0)
- 12346839

عند إيجاد Test Cases نلاحظ وجود مسار لا يمكن تحقيقه، حيث أن الذي يحكم المسارات هو الشروط والحلقات، وبالتالي لاختيار المدخلات ننظر لشرط دخول الحلقة والخروج منها.

للدخول إلى الحلقة يجب أن يكون مجموع $x + y > 0$ فالمسار 12346839 غير محقق حيث سيقابل كون x ضمن الشرط في العقدة 4 أصغر أو يساوي الصفر و y ضمن الشرط في العقدة 6 أصغر أو يساوي الصفر، وبالتالي مجموعهما أصغر أو يساوي الصفر، أي لم نستطيع دخول الحلقة من البداية، حيث لدخولها يجب أن يكون أحد الشرطين في 4 أو 6 محقق و بالتالي هذا مسار مستحيل و لن يتحقق أبداً.

نلتقاكم في محاضرات قادمة..

- انتهت المحاضرة -

